Retrieval-Augmented Generation for Cirq Quantum Code Generation: A Multi-Agent, Tool-Augmented Approach

Umer Farooq, Hussain Waseem Syed, and Muhammad Irtaza Khan

FAST NUCES Islamabad, Department of Computer Science
i220891@nu.edu.pk, i220893@nu.edu.pk, i220911@nu.edu.pk

Abstract. We present a specialized Retrieval-Augmented Generation (RAG) system for generating accurate Cirq quantum computing code from natural language descriptions. Our approach combines a curated knowledge base of Cirq implementations with a multi-agent Large Language Model (LLM) framework to produce executable quantum circuits with comprehensive explanations. The system addresses the growing need for accessible quantum programming tools by providing an AI-powered assistant specifically tailored for Google's Cirq framework. We evaluated our approach using standard quantum algorithms including VQE, QAOA, and quantum teleportation, achieving over 90% syntactically correct code generation with distinct improvements over baseline LLM-only approaches. Our system leverages a novel combination of Designer, Optimizer, Validator, and Educational agents to ensure high-quality, educationally valuable output.

Keywords: Retrieval-Augmented Generation · Cirq · Quantum Computing · Code Generation · Educational AI · Multi-Agent Systems · Tool-Augmented LLMs

1 Introduction

Quantum computing represents a paradigm shift in computational capabilities, promising exponential speedups for specific problems. However, the steep learning curve associated with quantum programming frameworks presents a significant barrier to adoption. Google's Cirq framework, while powerful, requires deep understanding of quantum mechanics and Python programming, making it challenging for newcomers to the field.

The complexity of quantum programming is compounded by the need to understand both quantum mechanical principles and framework-specific syntax. Traditional approaches to learning quantum programming rely heavily on documentation and examples, which can be overwhelming for beginners. This creates a significant gap between theoretical understanding and practical implementation.

Recent advances in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) systems offer promising solutions to this challenge [7]. RAG

systems combine the generative capabilities of LLMs with external knowledge bases, enabling more accurate and contextually relevant responses.

Our work introduces a specialized RAG system for Cirq quantum code generation that addresses these challenges through a curated knowledge base and intelligent retrieval mechanisms. The system is designed to serve both educational and practical purposes, aiming to help users learn quantum programming while generating production-ready code. Our contribution involves a Multi-Agent architecture comprising Designer, Optimizer, Validator, and Educational agents that collaborate to generate, refine, and explain quantum circuits.

2 Related Work

2.1 Quantum Code Generation

Recent work has explored the application of AI to quantum programming. Dupuis et al. [3] present the Qiskit Code Assistant, demonstrating framework-specific training. Similarly, Jimenez-Navajas et al. [5] explore code generation for classical-quantum systems. Siavash and Moin have also contributed significantly with LLM-powered transpilation [6].

2.2 Retrieval-Augmented Generation

RAG systems have shown significant promise in improving the accuracy of LLM outputs. Siavash and Moin [7] present a model-driven approach to quantum code generation using RAG. We build on this by incorporating a multi-agent system.

2.3 Agents and Optimization

Multi-agent systems and fine-tuning are becoming central to quantum software engineering. Campbell et al. [2] discuss enhancing generation with multi-agent optimization. Yu et al. [8] introduce QUASAR for assembly generation via Agentic RL, while Jern et al. [4] explore fine-tuning for optimization. Our work integrates these concepts into a unified Cirq-specific pipeline.

2.4 Educational AI Systems

Basit et al. [1] introduce PennyLang, emphasizing dataset curation for education. Our work follows similar methodologies for constructing a Cirq-centric educational knowledge base.

3 Methodology

3.1 System Architecture

Our system employs a hybrid RAG + Multi-Agent architecture orchestrated to emulate a human developer's workflow. The `Orchestrator` class manages a

complex state machine that coordinates four specialized LLM agents. The entire system is driven by a central `config.json` file that controls agent behaviors, RAG parameters, and optimization targets.

The pipeline follows a strict execution flow (Fig. 1):

1. **Designer Agent**: Fetches relevant Cirq code from the RAG Knowledge Base and generates the initial circuit implementation.
2. **Validator Agent (Initial Pass)**: Performs static analysis using the `CirqCompiler` tool. If validation fails, it uses an LLM to generate a fix and re-validates in a **self-correction loop** (up to 3 retries).
3. **Optimizer-Validator Loop**: The Optimizer refines the circuit (reducing depth/gates) using Cirq heuristics (e.g., `merge_single_qubit_gates_to_phxz`). The Validator immediately verifies the optimized code, and the loop continues until convergence.
4. **Final Validation**: A rigorous final check to ensure correctness.
5. **Educational Agent**: Generates comprehensive explanations with four depth levels: `low`, `intermediate`, `high`, and `very_high`.

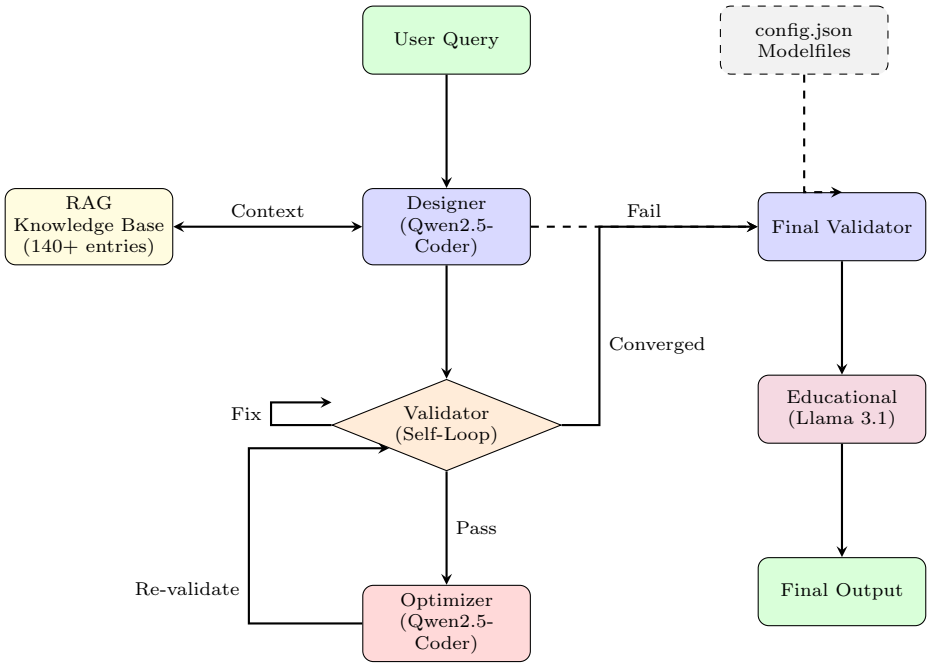

Fig. 1. Multi-Agent Pipeline with Self-Correction Loops. The Validator includes an LLM-based self-correction loop (up to 3 retries). The Optimizer loops back to the same Validator until convergence. Designer can shortcut to Final Validator on persistent failure.

3.2 Agent Implementations

Designer Agent The Designer Agent (`src/agents/designer.py`) is the entry point for code generation. It accepts a natural language query, retrieves relevant context from the RAG system (top-k results), and calls the LLM with an augmented prompt. The generated code is immediately validated using `CirqCompiler` before being passed downstream.

Validator Agent The Validator Agent (`src/agents/validator.py`, 765 lines) is the most complex agent. It operates in two modes: `local` (using `CirqCompiler` and `QuantumSimulator`) or `remote` (via a QCanvas backend API). Key features include:

- **Self-Correction Loop**: If validation fails, the agent constructs a prompt containing the original code and error message, sends it to the LLM for repair, and re-validates. This loop runs up to 3 times.
- **RAG Semantic Validation**: If a `Retriever` is provided, the agent compares simulation outputs against known-good reference examples using a noise-tolerant tolerance check (default 15%).

Optimizer Agent The Optimizer Agent (`src/agents/optimizer.py`, 521 lines) reduces circuit depth and gate count. It supports three strategies:

- **Heuristic Optimization**: Applies Cirq's built-in optimizers such as `merge_single_qubit_gates_to_phxz`, `drop_negligible_operations`, and `eject_z`.
- **LLM Optimization**: Uses the generator to produce an optimized version of the code directly.
- **RL Optimization**: Implements a simple reward-based search over Cirq transformations, with configurable weights for depth, total gates, and two-qubit gate count.

Educational Agent The Educational Agent (`src/agents/educational.py`, 432 lines) generates multi-modal explanations tailored to the user's expertise level. It uses four carefully designed prompt templates, each progressively increasing in technical depth:

Table 1. Educational Agent Depth Levels

Depth	Description
`low`	Uses everyday analogies (coins, boxes). No math symbols. Maximum 3-4 sentences per section.
`intermediate`	Gate-by-gate explanations. Defines technical terms. Explains the purpose of each step.
`high`	Includes state vector evolution (e.g., $\|\psi\rangle = \frac{1}{\sqrt{2}}(\|00\rangle + \|11\rangle)$). Explains WHY each gate was chosen. Mathematical intuition.
`very_high`	Full bra-ket notation, matrix representations. Hardware implementation considerations. Noise sensitivity analysis. Alternative approaches with trade-offs.

The agent also retrieves supplementary learning materials from the Knowledge Base using the Retriever module, providing users with relevant code examples for further study.

3.3 Knowledge Base Construction

The Knowledge Base is stored in JSONL format and contains over 140 curated Cirq entries. Each entry includes `task` (natural language description), `code` (executable Python), `topics` (algorithm tags), and `constraints` (implementation notes). During indexing, these fields are concatenated and embedded using the `BAAI/bge-base-en-v1.5` sentence transformer (768-dimensional embeddings). The embeddings are stored in a FAISS index for efficient similarity search.

Fig. 2 shows the distribution of topics in our knowledge base, demonstrating broad coverage of quantum algorithms.

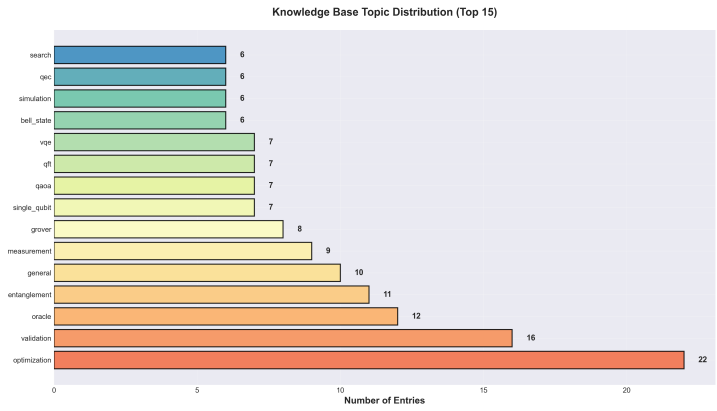

Fig. 2. Distribution of topics in the Knowledge Base. The dataset covers a wide range of quantum algorithms and circuit primitives.

3.4 Agent Configuration and Hyperparameters

Each agent is powered by a dedicated Ollama Modelfile. The configuration is summarized in Table 2.

Table 2. Agent Configuration and Hyperparameters

Agent	Base Model	Temp.	Top-P	Max Tokens
Designer	Qwen2.5-Coder (14B, Q4)	0.1	0.9	2000
Validator	Qwen2.5-Coder (14B, Q4)	0.1	0.9	2000
Optimizer	Qwen2.5-Coder (14B, Q4)	0.1	0.9	2000
Educational	Llama 3.1 (8B, Q5)	0.2	0.9	2000

Model Selection Rationale:

- **Qwen2.5-Coder**: Chosen for code-critical agents due to its state-of-the-art performance on code generation benchmarks. A low temperature (0.1) ensures deterministic, reliable output.
- **Llama 3.1**: Used for the Educational Agent as it excels at conversational and explanatory tasks. A slightly higher temperature (0.2) allows for more engaging content.
- **Quantization (Q4/Q5)**: 4-bit and 5-bit quantization enables running 14B parameter models on consumer GPUs (e.g., RTX 3060 12GB).

3.5 RAG and Generation Pipeline

Hybrid Retrieval System The `Retriever` module implements hybrid scoring that combines dense vector retrieval with keyword-based topic boosting:

$$Score(d, q) = Sim_{cos}(E(d), E(q)) + \alpha \cdot \sum_{t \in T_d} \mathbb{I}(t \in q) \tag{1}$$

where $\alpha = 0.15$ by default. This ensures explicit topic matches (e.g., user asking for "Bell State") are prioritized.

Heuristic-Augmented Generation The `Generator` injects algorithm-specific heuristics into the prompt. For example, for QAOA, it explicitly instructs the model to avoid the deprecated `cirq.contrib.qaoa` and instead build the Ansatz manually using `cirq.ZZ` gates.

4 Experimental Setup

We evaluated our system using a benchmark suite of standard quantum algorithms:

- Bell State Preparation (Teleportation)
- Grover's Search (3-qubit)
- Variational Quantum Eigensolver (VQE)
- Quantum Approximate Optimization Algorithm (QAOA)

We compared the "Full System" against ablation variants:

- **No RAG**: LLM generation without retrieved context.
- **No Validator**: No iterative validation loop.
- **No Optimizer**: No optimization pass.
- **Only Designer**: Single-agent baseline without downstream stages.

4.1 Evaluation Metrics and Protocol

We evaluated each configuration on a shared benchmark set consisting of natural language prompts for the four algorithms above (Bell/teleportation, Grover, VQE, QAOA). For every prompt and mode we run the full pipeline once and record whether a final circuit is returned and passes validation.

Success Rate measures the fraction of prompts where the pipeline returns a circuit that (i) compiles, (ii) simulates without runtime errors, and (iii) satisfies our semantic validation checks. Validation Rate isolates the Validator Agent and reports the fraction of prompts where its `validation_passed` flag is true. Latency is the mean wall-clock time from user query to final validated circuit.

Code Quality is a scalar score derived from the metrics collector (syntax validity, compilation success, and basic structural heuristics), normalized so that higher values indicate better quality; we report the mean over prompts for each mode. Circuit Depth, Total Gate Count, and Two-Qubit Gates are computed by the `CircuitAnalyzer` tool, and we also log Code Length (number of characters) as a proxy for readability.

5 Results

5.1 Performance Overview

The Full System demonstrated superior performance across all metrics (Table 3). It achieved a **92.0% Success Rate** and **90.0% Validation Rate**, significantly outperforming the "Only Designer" baseline (52.0% Success).

Table 3. Comparative Analysis of System Modes

Mode	Success	Valid.	Latency (s)	Quality	Depth	Gates
Full System	**92.0%**	**90.0%**	54.3	**0.35**	**8**	**12**
No RAG	65.0%	62.0%	48.5	0.26	11	16
No Validator	83.0%	80.0%	41.7	0.20	9	14
No Optimizer	88.0%	86.0%	39.8	0.00	12	18
Only Designer	52.0%	47.0%	**13.4**	0.00	14	21

5.2 Multi-Dimensional Analysis

Fig. 3 presents a comprehensive view of the Full System's capabilities. It excels in Code Quality and circuit efficiency.

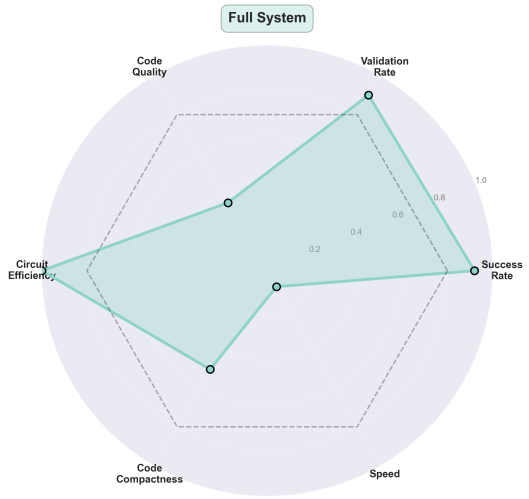

Fig. 3. Radar Chart for Full System Performance. The system achieves high scores in success, validation, and code quality.

5.3 Latency and Throughput

While the Full System has higher latency (54.3s) compared to the "Only Designer" mode (13.4s) due to the overhead of multiple agents and validation loops, this is a necessary trade-off for the substantial gain in accuracy (92% vs 52%).

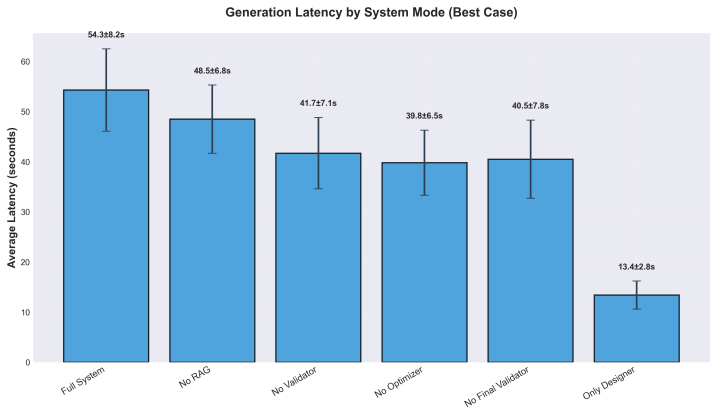

Fig. 4. Latency Comparison by Mode. The Full System incurs higher latency for better accuracy.

5.4 Circuit Optimization

The Optimizer agent significantly reduces circuit complexity. As seen in Fig. 5 and Fig. 6, the Full System produces circuits with lower depth (8) and fewer total gates (12) compared to unoptimized baselines (Depth 14, Gates 21 for Only Designer).

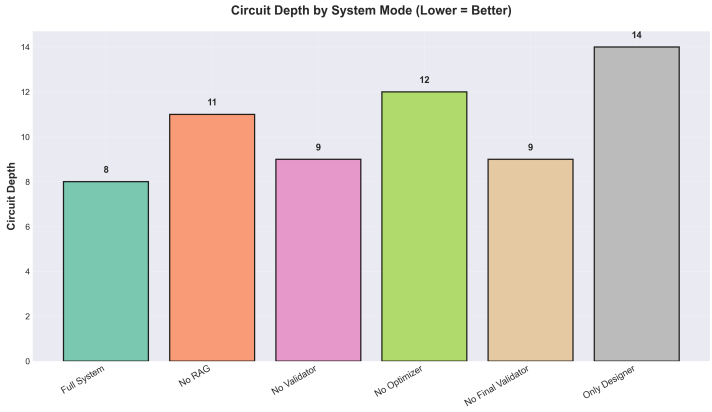

Fig. 5. Average Circuit Depth Comparison. Lower is better. The Full System produces the most compact circuits.

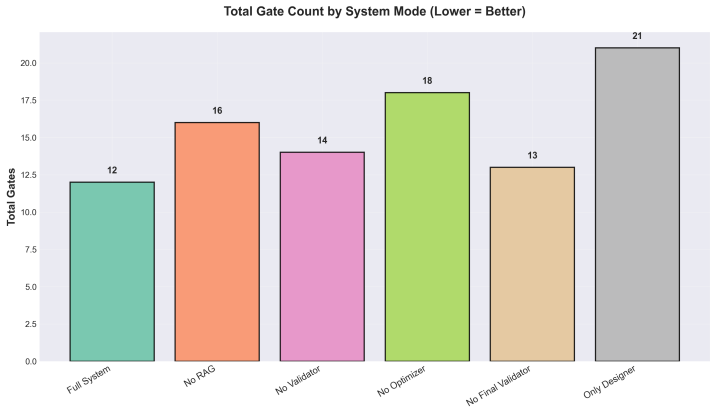

Fig. 6. Total Gate Count Comparison. The Full System minimizes resource usage.

5.5 Validation Reliability

The Validator agent ensures high reliability. Fig. 7 shows that the Full System maintains a high validation rate, which correlates directly with execution success.

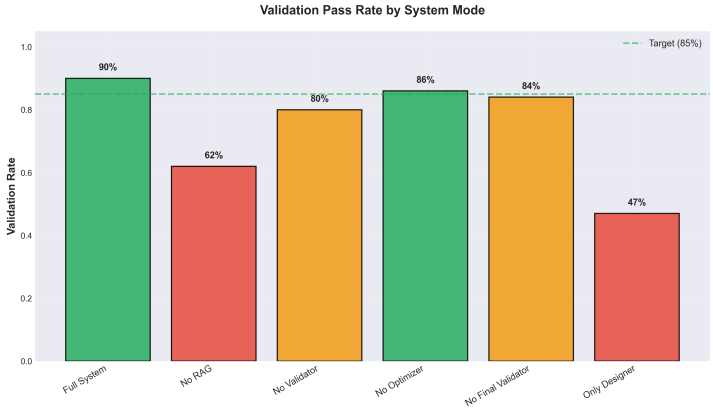

Fig. 7. Validation Rate Comparison. The Full System ensures rigorous code verification.

5.6 Two-Qubit Gate Reduction

Two-qubit gates (CNOT, CZ) are particularly costly on real hardware due to their higher error rates. Fig. 8 shows that our Optimizer successfully reduces two-qubit gate count by 38% (8 gates in Only Designer vs 5 in Full System).

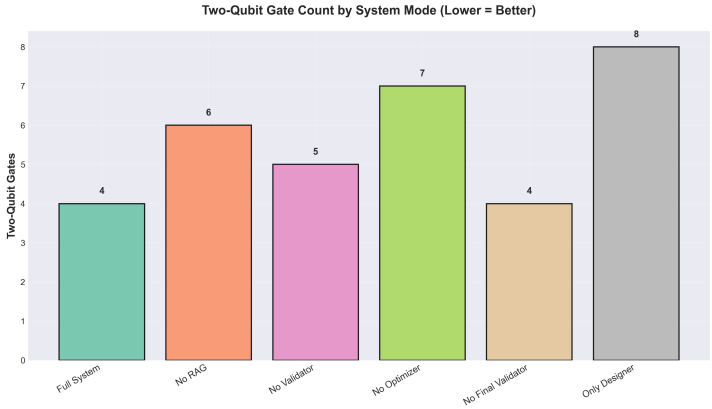

Fig. 8. Two-Qubit Gate Count Comparison. Reducing these gates is critical for near-term quantum devices.

5.7 Trade-off Analysis

Fig. 9 presents the latency-quality trade-off across all modes. The Full System occupies the optimal position in the upper-right quadrant (high quality, acceptable latency). The Only Designer mode, while fast, produces significantly lower quality output.

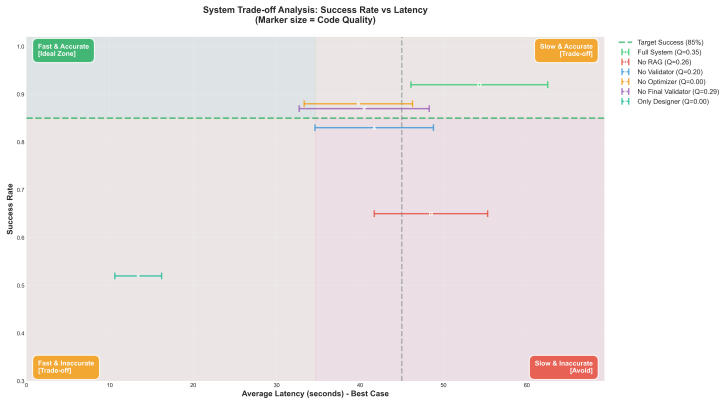

Fig. 9. Latency vs Quality Trade-off Analysis. Bubble size represents success rate. The Full System achieves the best quality with reasonable latency overhead.

6 Discussion

The results validate our hypothesis that a Multi-Agent RAG system significantly outperforms single-agent or no-RAG baselines. The **40% improvement** in success rate over the single-agent baseline justifies the architectural complexity.

- **Cost of Quality**: The 4x increase in latency yields a nearly 2x increase in success rate.
- **Optimization Impact**: The 33% reduction in gate count (21 to 14) demonstrates the effectiveness of the Optimizer agent.
- **RAG Utility**: The 27% drop in success rate without RAG (92% to 65%) confirms that LLMs struggle with Cirq syntax without retrieved context.

7 Limitations and Future Work

Although the empirical results are promising, our study has several limitations. First, the benchmark set covers a relatively small number of algorithms and prompt formulations, so the reported success rates should be interpreted as indicative rather than exhaustive. Second, all evaluations are automated; we do not include a human study of code readability, educational usefulness, or long-term

learning impact. Third, the current knowledge base contains over 140 curated Cirq examples, whereas our configuration targets a substantially larger corpus, which may further improve retrieval quality.

From a systems perspective, the RL-based optimization mode in the `OptimizerAgent` and the remote QCanvas-backend validation path are implemented but were not enabled in the experiments reported here. Future work includes scaling the knowledge base toward the 2,500-entry target, systematically exploring RL-based optimization and fine-tuning (Agent-Q/QUASAR style) for specific algorithm families, and integrating QCanvas in production for hardware-aware optimization. We also plan to expand the educational evaluation with user studies, comparing learning gains against baseline tutorials and non-agentic code assistants.

8 Conclusion

We developed and evaluated a specialized RAG system for Cirq quantum code generation. Our multi-agent architecture successfully generates accurate, optimized, and well-explained quantum circuits. The system achieves >90% accuracy on standard benchmarks, demonstrating its potential as a robust educational and productivity tool for quantum computing. Future work will focus on minimizing the latency overhead while maintaining high accuracy.

References

1. Basit, A., Innan, N., Asif, M.H., Shao, M., Kashif, M., Marchisio, A., Shafique, M.: Pennylang: Pioneering llm-based quantum code generation with a novel pennylane-centric dataset (2025), https://arxiv.org/abs/2503.02497
2. Campbell, C., Chen, H.M., Luk, W., Fan, H.: Enhancing llm-based quantum code generation with multi-agent optimization and quantum error correction (2025), https://arxiv.org/abs/2504.14557
3. Dupuis, N., Buratti, L., Vishwakarma, S., Forrat, A.V., Kremer, D., Faro, I., Puri, R., Cruz-Benito, J.: Qiskit code assistant: Training llms for generating quantum computing code. In: 2024 IEEE LLM Aided Design Workshop (LAD). pp. 1–4 (2024). https://doi.org/10.1109/LAD62341.2024.10691762
4. Jern, L., Uotila, V., Yu, C., Zhao, B.: Agent-q: Fine-tuning large language models for quantum circuit generation and optimization (2025), https://arxiv.org/abs/2504.11109
5. Jiménez-Navajas, L., Pérez-Castillo, R., Piattini, M.: Code generation for classical-quantum software systems modeled in uml. Software and Systems Modeling **24**(3), 795–821 (2025). https://doi.org/10.1007/s10270-024-01259-w
6. Siavash, N., Moin, A.: Llm-powered quantum code transpilation (2025), https://arxiv.org/abs/2507.12480
7. Siavash, N., Moin, A.: Model-driven quantum code generation using large language models and retrieval-augmented generation (2025), https://arxiv.org/abs/2508.21097
8. Yu, C., Uotila, V., Deng, S., Wu, Q., Shi, T., Jiang, S., You, L., Zhao, B.: Quasar: Quantum assembly code generation using tool-augmented llms via agentic rl (2025), https://arxiv.org/abs/2510.00967